

Table of Contents

Lab Manual: Alert Receiving a Message (SMS)

Project: Alert-Receiving-A-Msg

Package: com.example.alert_receiving_a_msg

Platform: Android (Java)

Minimum SDK: 26 | **Target SDK:** 36

Document version: 2.0 (includes full source listings)

Table of Contents

1. [Introduction](#)
 2. [Project Structure \(Structural View\)](#)
 3. [Application Flow \(Top to Bottom\)](#)
 4. [Configuration Files](#)
 5. [Resource Files \(XML — values/\)](#)
 6. [Layout and Navigation XML](#)
 7. [Java Source Files \(Top-to-Bottom Flow\)](#)
 8. [Complete Lifecycle Flow \(Summary\)](#)
 9. [File Relationship Diagram](#)
 10. [Lab Exercises for Students](#)
 11. [Prerequisites Checklist](#)
 12. [Troubleshooting](#)
 13. [Appendix D — Complete Source Code](#)
-

1. Introduction

1.1 Purpose of the Lab

This laboratory exercise introduces **Broadcast Receivers**, **runtime permissions**, and **SMS handling** on Android. Students build and study an application that:

- Requests permission to **receive** and **send** SMS messages.

- **Listens** for incoming SMS broadcasts while the app is in the foreground.
- Displays an **AlertDialog** when a message arrives.
- Provides a simple UI to **send** an SMS from the same device (useful for emulator testing).

1.2 Learning Objectives

Objective	Description
LO1	Explain how Android delivers system events (such as incoming SMS) via Intents and BroadcastReceivers .
LO2	Register a receiver dynamically in an Activity lifecycle (onResume / onPause).
LO3	Request dangerous permissions (RECEIVE_SMS, SEND_SMS) at runtime on Android 6.0+.
LO4	Parse SMS PDUs from a broadcast Intent using SmsMessage.
LO5	Use View Binding to access UI widgets from Java code.
LO6	Read and explain XML layouts , manifest , and Gradle configuration.

1.3 How to Use This Manual

- Sections **1–12** explain concepts and flow **top to bottom** (structure → flow → config → resources → layouts → Java → lifecycle).
 - **Section 13 / Appendix D** contains the **complete source code** for every XML and Java file (nothing omitted).
 - Match each listing to its path under app/src/main/ when working in Android Studio.
-

2. Project Structure (Structural View)

2.1 Directory Tree

```
AlertReceivingAMsg/
|-- settings.gradle.kts
|-- build.gradle.kts
|-- gradle/libs.versions.toml
```

```

|-- LAB_MANUAL.md
|-- scripts/build_lab_manual.py
`-- app/
    |-- build.gradle.kts
    `-- src/
        |-- main/
        | |-- AndroidManifest.xml
        | |-- java/.../MainActivity.java      [ACTIVE]
        | |-- java/.../MessageReceiver.java   [ACTIVE]
        | |-- java/.../FirstFragment.java     [template]
        | |-- java/.../SecondFragment.java    [template]
        | `-- res/
        |     |-- layout/activity_main.xml     [ACTIVE]
        |     |-- layout/content_main.xml     [template]
        |     |-- layout/fragment_*.xml       [template]
        |     |-- navigation/nav_graph.xml    [template]
        |     |-- menu/menu_main.xml          [template]
        |     |-- values/                     [themes, strings, colors]
        |     `-- xml/backup_rules.xml
        |-- test/ExampleUnitTest.java
        `-- androidTest/ExampleInstrumentedTest.java

```

ACTIVE = used when you run the app today. **template** = leftover Navigation demo from Android Studio (included in full for study).

2.2 Module Responsibilities

Component	Role
MainActivity	Launcher UI, permissions, send SMS, register/unregister receiver
MessageReceiver	Parses SMS_RECEIVED broadcast PDUs
activity_main.xml	Toolbar, status, phone/message fields, Send button
AndroidManifest.xml	Permissions, declares launcher activity

3. Application Flow (Top to Bottom)

3.1 Execution Order

1. User launches app → system reads **AndroidManifest.xml** → starts **MainActivity**.
2. **onCreate**: View Binding inflates **activity_main.xml**; permissions requested; welcome **AlertDialog**; Send button wired.
3. **onResume**: `MessageReceiver.addListener(this)`; `registerReceiver` for `SMS_RECEIVED`.
4. **Send path**: User fills `edit_phone`, `edit_message` → `SmsManager.sendTextMessage()`.
5. **Receive path**: Telephony broadcasts intent → **MessageReceiver.onReceive** → PDU → `onMessageReceived` → **AlertDialog**.
6. **onPause**: `unregisterReceiver`; `addListener(null)`.

3.2 Data Flow

Step	From	To	Data
1	User	Layout	Phone, message text
2	MainActivity	SmsManager	Outgoing SMS
3	System	MessageReceiver	Intent extras: pdus, format
4	MessageReceiver	MainActivity	Message body string
5	MainActivity	User	AlertDialog, Toasts

4. Configuration Files

Configuration defines **what the app is allowed to do** and **how it is built**. Full source appears in **Appendix D**; summaries below.

4.1 AndroidManifest.xml

Element	Purpose
<code>RECEIVE_SMS</code>	Receive SMS broadcasts
<code>SEND_SMS</code>	Send SMS via <code>SmsManager</code>

Element	Purpose
telephony (not required)	Install on devices without phone hardware
MainActivity + LAUNCHER	App entry point

MessageReceiver is **not** in the manifest; it is registered in code in onResume.

4.2 Gradle Files

File	Purpose
settings.gradle.kts	Project name; includes :app module
build.gradle.kts (root)	Android plugin declaration
app/build.gradle.kts	minSdk 26, targetSdk 36, viewBinding true , dependencies
gradle/libs.versions.toml	Centralized library versions (Material, Navigation, JUnit)

4.3 Key Build Settings

- **View Binding** generates ActivityMainBinding from activity_main.xml.
- **Java 11** for source/target compatibility.
- Navigation libraries are on the classpath for template fragments only.

Full code: See Appendix D — AndroidManifest.xml, settings.gradle.kts, build.gradle.kts, app/build.gradle.kts, libs.versions.toml.

5. Resource Files (XML — values/)

5.1 strings.xml

Resource	Used by
app_name	Launcher label (manifest)
action_settings	menu_main.xml
first_fragment_label, second_fragment_label	Navigation graph
next, previous	Fragment buttons
lorem_ipsum	Placeholder text in fragment layouts

5.2 colors.xml

black (#FF000000), white (#FFFFFFFF) — available for custom theming.

5.3 themes.xml (values, values-night, values-v23)

Style	Parent	Notes
Base.Theme.AlertReceivin gAMsg	Material3 DayNight NoActionBar	Light/dark base
Theme.AlertReceivingAM sg	Base theme	Applied in manifest
values-v23 variant	Transparent status/navigation bars	Edge-to-edge support

5.4 dims.xml (and qualifiers)

File	fab_margin	When applied
values/dimens.xml	16dp	Default
values-land/	48dp	Landscape
values-w600dp/	48dp	Width >= 600dp
values-w1240dp/	200dp	Large width (e.g. tablets)

Full code: See Appendix D — all res/values* XML files.

6. Layout and Navigation XML

6.1 activity_main.xml (ACTIVE)

View	ID	Role
MaterialToolbar	toolbar	Title “Alert Receiver”
TextView	status_text	Listening status
EditText	edit_phone	Phone input
EditText	edit_message	Message body
Button	btn_send	Calls sendSms()

6.2 Template layouts (not used by current MainActivity)

File	Purpose
content_main.xml	Hosts NavHostFragment + nav_graph
fragment_first.xml / fragment_second.xml	Navigation demo screens
nav_graph.xml	First <-> Second fragment actions
menu_main.xml	Overflow Settings item

6.3 Backup XML

- backup_rules.xml — full backup rules (API < 31).
- data_extraction_rules.xml — cloud backup / device transfer (API 31+).

Full code: See Appendix D — all res/layout, res/navigation, res/menu, res/xml files.

7. Java Source Files (Top-to-Bottom Flow)

Study Java files in this order:

7.1 MessageReceiver.java

- Extends BroadcastReceiver.
- Defines MessageListener callback interface.
- onReceive: reads pdus from intent, builds SmsMessage, calls listener or Toast.

7.2 MainActivity.java

- Implements MessageListener.
- onCreate: binding, toolbar, permissions, welcome dialog, Send listener.
- sendSms: validation + SmsManager.
- onResume / onPause: register/unregister receiver; bind/unbind listener.
- onMessageReceived: incoming **AlertDialog**.

7.3 FirstFragment.java / SecondFragment.java

- Navigation Component template; View Binding; navigate via NavHostFragment.
- Not started by current manifest (no reference from MainActivity).

7.4 Test classes

- ExampleUnitTest — JVM test on host (2+2=4).

- ExampleInstrumentedTest — verifies package name on device.

Full code: See Appendix D — all .java files with line-for-line listings.

8. Complete Lifecycle Flow (Summary)

[Not running]

| onCreate (inflate layout, permissions, UI setup)

v

[Created] --> onStart --> onResume

|

REGISTER MessageReceiver

BIND MessageListener

|

+-----+-----+

| User sends SMS | Incoming SMS |

+-----+-----+

|

onPause

|

UNREGISTER receiver

UNBIND listener

v

onStop / onDestroy

Why onPause/onResume? Avoids leaks; AlertDialog needs active Activity; no background SMS in this lab design.

9. File Relationship Diagram

AndroidManifest.xml

|

+--> MainActivity.java <-- View Binding -- activity_main.xml

|

|

+--> registers --> MessageReceiver.java

|

|

+<----- onMessageReceived (listener)

|

+--> @string/app_name <-- strings.xml

+--> @style/Theme... <-- themes.xml

[SMS_RECEIVED broadcast] --> MessageReceiver.onReceive()

Template chain (inactive in current run):

content_main.xml --> nav_graph.xml --> FirstFragment / SecondFragment
|
fragment_first.xml / fragment_second.xml

10. Lab Exercises for Students

1. **Trace the broadcast** — Sequence diagram: User, Telephony, MessageReceiver, MainActivity.
2. **onRequestPermissionsResult** — Handle denial; disable Send until granted.
3. **Show sender number** — Pass getDisplayOriginatingAddress() to the AlertDialog.
4. **Logcat** — Add Log.d in onReceive and onMessageReceived; submit screenshot.
5. **Validation** — Phone length and 160-character message limit with counter TextView.
6. **Research** — Manifest vs dynamic receiver registration (half page).
7. **Code reading** — Using only Appendix D, explain lines 26–38 in MessageReceiver.java without running the app.

Submission Checklist

- ☐ Lab report (Exercises 1, 6, 7)
 - ☐ Screenshots: welcome dialog, incoming alert, send success Toast
 - ☐ Logcat snippet
 - ☐ Optional: annotated printout of MainActivity.java from this manual
-

11. Prerequisites Checklist

Item	Requirement
Android Studio	Recent stable release
JDK	11+
SDK	API 36 platform installed
Emulator	Google APIs image; Extended Controls -> Phone for

Item	Requirement
	SMS test
Knowledge	Activity lifecycle, basic XML, Java interfaces

Permissions

Permission	Level	Purpose
RECEIVE_SMS	Dangerous	Incoming SMS
SEND_SMS	Dangerous	Outgoing SMS

12. Troubleshooting

Problem	Solution
No incoming alert	App must be foreground (onResume); grant SMS permissions
Send fails	Grant SEND_SMS; fill both fields; use emulator phone number
SecurityException on registerReceiver	Project uses RECEIVER_EXPORTED on API 33+
Build errors	Sync Gradle; enable View Binding in app/build.gradle.kts
Template confusion	Only activity_main + MainActivity + MessageReceiver are on the live path

Emulator SMS Test

1. Run app; grant permissions.
2. Emulator **Extended Controls** (three dots) -> **Phone**.
3. **To** = emulator number; enter message -> **Send Message**.
4. Confirm **Incoming Message Alert** dialog.

Appendix D — Complete Source Code (Project Files)

This appendix contains the **full, current source** of every XML and Java file in the lab project. Study files in the order listed (top to bottom).

AndroidManifest.xml

Path: app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.RECEIVE_SMS" />
    <uses-permission android:name="android.permission.SEND_SMS" />
    <uses-feature android:name="android.hardware.telephony"
android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AlertReceivingAMsg">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.AlertReceivingAMsg">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

settings.gradle.kts

Path: settings.gradle.kts

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "Alert-Receiving-A-Msg"
include(":app")
```

build.gradle.kts (root)

Path: build.gradle.kts

// Top-level build file where you can add configuration options common to all sub-projects/modules.

```
plugins {
    alias(libs.plugins.android.application) apply false
}
```

app/build.gradle.kts

Path: app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.alert_receiving_a_msg"
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }
}

defaultConfig {
    applicationId = "com.example.alert_receiving_a_msg"
    minSdk = 26
    targetSdk = 36
    versionCode = 1
    versionName = "1.0"

    testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
}

buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}

compileOptions {
    sourceCompatibility = JavaVersion.VERSION_11
    targetCompatibility = JavaVersion.VERSION_11
}

buildFeatures {
    viewBinding = true
}

dependencies {
    implementation(libs.activity.ktx)
```

```
implementation(libs.appcompat)
implementation(libs.constraintlayout)
implementation(libs.material)
implementation(libs.navigation.fragment)
implementation(libs.navigation.ui)
testImplementation(libs.junit)
androidTestImplementation(libs.espresso.core)
androidTestImplementation(libs.ext.junit)
}
```

gradle/libs.versions.toml

Path: gradle/libs.versions.toml

```
[versions]
agp = "9.2.1"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.7.1"
material = "1.14.0"
constraintlayout = "2.2.1"
activityKtx = "1.13.0"
navigationFragment = "2.9.8"
navigationUi = "2.9.8"

[libraries]
junit = { group = "junit", name = "junit", version.ref = "junit" }
ext-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
material = { group = "com.google.android.material", name = "material", version.ref = "material" }
constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout" }
activity-ktx = { group = "androidx.activity", name = "activity-ktx", version.ref = "activityKtx" }
navigation-fragment = { group = "androidx.navigation", name = "navigation-fragment", version.ref = "navigationFragment" }
navigation-ui = { group = "androidx.navigation", name = "navigation-ui", version.ref = "navigationUi" }
```

[plugins]

android-application = { id = "com.android.application", version.ref = "agp" }

res/values/strings.xml

Path: app/src/main/res/values/strings.xml

<resources>

<string name="app_name">Alert-Receiving-A-Msg</string>

<string name="action_settings">Settings</string>

<!-- Strings used for fragments for navigation -->

<string name="first_fragment_label">First Fragment</string>

<string name="second_fragment_label">Second Fragment</string>

<string name="next">Next</string>

<string name="previous">Previous</string>

<string name="lorem_ipsum">

 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam in scelerisque sem.
Mauris

 volutpat, dolor id interdum ullamcorper, risus dolor egestas lectus, sit amet mattis
purus

 dui nec risus. Maecenas non sodales nisi, vel dictum dolor. Class aptent taciti
sociosqu ad

 litora torquent per conubia nostra, per inceptos himenaeos. Suspendisse blandit
eleifend

 diam, vel rutrum tellus vulputate quis. Aliquam eget libero aliquet, imperdiet nisl a,
ornare ex. Sed rhoncus est ut libero porta lobortis. Fusce in dictum tellus.\n\n
 Suspendisse interdum ornare ante. Aliquam nec cursus lorem. Morbi id magna felis.

Vivamus

 egestas, est a condimentum egestas, turpis nisl iaculis ipsum, in dictum tellus dolor
sed

 neque. Morbi tellus erat, dapibus ut sem a, iaculis tincidunt dui. Interdum et
malesuada

 fames ac ante ipsum primis in faucibus. Curabitur et eros porttitor, ultricies urna
vitae,

 molestie nibh. Phasellus at commodo eros, non aliquet metus. Sed maximus nisl nec
dolor

 bibendum, vel congue leo egestas.\n\n

 Sed interdum tortor nibh, in sagittis risus mollis quis. Curabitur mi odio,
condimentum sit

 amet auctor at, mollis non turpis. Nullam pretium libero vestibulum, finibus orci vel,
molestie quam. Fusce blandit tincidunt nulla, quis sollicitudin libero facilisis et.

Integer

interdum nunc ligula, et fermentum metus hendrerit id. Vestibulum lectus felis,
dictum at

lacinia sit amet, tristique id quam. Cras eu consequat dui. Suspendisse sodales nunc
ligula,

in lobortis sem porta sed. Integer id ultrices magna, in luctus elit. Sed a pellentesque
est.\n\n

Aenean nunc velit, lacinia sed dolor sed, ultrices viverra nulla. Etiam a venenatis
nibh.

Morbi laoreet, tortor sed facilisis varius, nibh orci rhoncus nulla, id elementum leo
dui

non lorem. Nam mollis ipsum quis auctor varius. Quisque elementum eu libero sed
commodo. In

eros nisl, imperdiet vel imperdiet et, scelerisque a mauris. Pellentesque varius ex
nunc,

quis imperdiet eros placerat ac. Duis finibus orci et est auctor tincidunt. Sed non
viverra

ipsum. Nunc quis augue egestas, cursus lorem at, molestie sem. Morbi a consectetur
ipsum, a

placerat diam. Etiam vulputate dignissim convallis. Integer faucibus mauris sit amet
finibus

convallis.\n\n

Phasellus in aliquet mi. Pellentesque habitant morbi tristique senectus et netus et
malesuada fames ac turpis egestas. In volutpat arcu ut felis sagittis, in finibus massa
gravida. Pellentesque id tellus orci. Integer dictum, lorem sed efficitur ullamcorper,
libero justo consectetur ipsum, in mollis nisl ex sed nisl. Donec maximus

ullamcorper

sodales. Praesent bibendum rhoncus tellus nec feugiat. In a ornare nulla. Donec
rhoncus

libero vel nunc consequat, quis tincidunt nisl eleifend. Cras bibendum enim a justo
luctus

vestibulum. Fusce dictum libero quis erat maximus, vitae volutpat diam dignissim.

</string>

</resources>

res/values/colors.xml

Path: app/src/main/res/values/colors.xml

<?xml version="1.0" encoding="utf-8"?>

<resources>

<color name="black">#FF000000</color>

<color name="white">#FFFFFFFF</color>

</resources>

res/values/themes.xml

Path: app/src/main/res/values/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.AlertReceivingAMsg"
parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.AlertReceivingAMsg"
parent="Base.Theme.AlertReceivingAMsg" />
</resources>
```

res/values/dimens.xml

Path: app/src/main/res/values/dimens.xml

```
<resources>
    <dimen name="fab_margin">16dp</dimen>
</resources>
```

res/values-night/themes.xml

Path: app/src/main/res/values-night/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.AlertReceivingAMsg"
parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your dark theme here. -->
        <!-- <item name="colorPrimary">@color/my_dark_primary</item> -->
    </style>
</resources>
```

res/values-v23/themes.xml

Path: app/src/main/res/values-v23/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Theme.AlertReceivingAMsg" parent="Base.Theme.AlertReceivingAMsg">
        <!-- Transparent system bars for edge-to-edge. -->
        <item name="android:navigationBarColor">@android:color/transparent</item>
```

```
<item name="android:statusBarColor">@android:color/transparent</item>
<item name="android:windowLightStatusBar"?attr/isLightTheme</item>
</style>
</resources>
```

res/values-land/dimens.xml

Path: app/src/main/res/values-land/dimens.xml

```
<resources>
  <dimen name="fab_margin">48dp</dimen>
</resources>
```

res/values-w600dp/dimens.xml

Path: app/src/main/res/values-w600dp/dimens.xml

```
<resources>
  <dimen name="fab_margin">48dp</dimen>
</resources>
```

res/values-w1240dp/dimens.xml

Path: app/src/main/res/values-w1240dp/dimens.xml

```
<resources>
  <dimen name="fab_margin">200dp</dimen>
</resources>
```

res/layout/activity_main.xml

Path: app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="16dp"
  android:gravity="center_horizontal">

  <com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    app:title="Alert Receiver" />
</com.google.android.material.appbar.AppBarLayout>
```

```
<TextView
    android:id="@+id/status_text"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="50dp"
    android:text="Status: Listening for Messages..."
    android:textSize="18sp"
    android:textStyle="bold" />
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="To test, send an SMS from the Emulator Controls."
    android:textAlignment="center" />
```

```
<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:layout_marginVertical="30dp"
    android:background="#CCCCCC" />
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Send a Message"
    android:textSize="16sp"
    android:textStyle="bold" />
```

```
<EditText
    android:id="@+id/edit_phone"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Phone Number"
    android:inputType="phone" />
```

```
<EditText
    android:id="@+id/edit_message"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Message Content"
    android:inputType="textMultiLine" />
```

```
<Button
    android:id="@+id/btn_send"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:text="Send SMS" />
```

```
</LinearLayout>
```

res/layout/content_main.xml

Path: app/src/main/res/layout/content_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment_content_main"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

res/layout/fragment_first.xml

Path: app/src/main/res/layout/fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FirstFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/next"
            app:layout_constraintBottom_toTopOf="@id/textview_first"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/textview_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_first" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

res/layout/fragment_second.xml

Path: app/src/main/res/layout/fragment_second.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".SecondFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/previous"
            app:layout_constraintBottom_toTopOf="@id/textview_second"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/textview_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_second" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>

```

res/navigation/nav_graph.xml

Path: app/src/main/res/navigation/nav_graph.xml

```

<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"

```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/nav_graph"
app:startDestination="@id/FirstFragment">

<fragment
    android:id="@+id/FirstFragment"
    android:name="com.example.alert_receiving_a_msg.FirstFragment"
    android:label="@string/first_fragment_label"
    tools:layout="@layout/fragment_first">

    <action
        android:id="@+id/action_FirstFragment_to_SecondFragment"
        app:destination="@id/SecondFragment" />
</fragment>
<fragment
    android:id="@+id/SecondFragment"
    android:name="com.example.alert_receiving_a_msg.SecondFragment"
    android:label="@string/second_fragment_label"
    tools:layout="@layout/fragment_second">

    <action
        android:id="@+id/action_SecondFragment_to_FirstFragment"
        app:destination="@id/FirstFragment" />
</fragment>
</navigation>

```

res/menu/menu_main.xml

Path: app/src/main/res/menu/menu_main.xml

```

<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="com.example.alert_receiving_a_msg.MainActivity">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>

```

res/xml/backup_rules.xml

Path: app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
  Sample backup rules file; uncomment and customize as necessary.
  See https://developer.android.com/guide/topics/data/autobackup
  for details.
  Note: This file is ignored for devices older than API 31
  See https://developer.android.com/about/versions/12/backup-restore
-->
<full-backup-content>
  <!--
  <include domain="sharedpref" path="."/>
  <exclude domain="sharedpref" path="device.xml"/>
  -->
</full-backup-content>
```

res/xml/data_extraction_rules.xml

Path: app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
  Sample data extraction rules file; uncomment and customize as necessary.
  See https://developer.android.com/about/versions/12/backup-restore#xml-changes
  for details.
-->
<data-extraction-rules>
  <cloud-backup>
    <!-- TODO: Use <include> and <exclude> to control what is backed up.
    <include .../>
    <exclude .../>
    -->
  </cloud-backup>
  <!--
  <device-transfer>
    <include .../>
    <exclude .../>
  </device-transfer>
  -->
</data-extraction-rules>
```

MainActivity.java

Path: app/src/main/java/com/example/alert_receiving_a_msg/MainActivity.java


```

package com.example.alert_receiving_a_msg;

import android.Manifest;
import android.content.IntentFilter;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.telephony.SmsManager;
import android.view.View;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.example.alert_receiving_a_msg.databinding.ActivityMainBinding;

public class MainActivity extends AppCompatActivity implements
    MessageReceiver.MessageListener {

    private ActivityMainBinding binding;
    private MessageReceiver messageReceiver;
    private static final int SMS_PERMISSION_CODE = 100;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);

        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        setSupportActionBar(binding.toolbar);

        ViewCompat.setOnApplyWindowInsetsListener(binding.main, (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,

```

```

systemBars.bottom);
    return insets;
});

checkPermissions();

messageReceiver = new MessageReceiver();

// Show a "Welcome" alert as requested
new AlertDialog.Builder(this)
    .setTitle("App Started")
    .setMessage("The application is now listening for incoming messages. Use the
form below to send a message.")
    .setPositiveButton("OK", null)
    .show();

binding.btnSend.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        sendSms();
    }
});
}

private void checkPermissions() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.RECEIVE_SMS) !=
PackageManager.PERMISSION_GRANTED ||
        ContextCompat.checkSelfPermission(this, Manifest.permission.SEND_SMS) !=
PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.RECEIVE_SMS,
Manifest.permission.SEND_SMS},
            SMS_PERMISSION_CODE);
    }
}

private void sendSms() {
    String phone = binding.editPhone.getText().toString();
    String message = binding.editMessage.getText().toString();

    if (phone.isEmpty() || message.isEmpty()) {

```

```

        Toast.makeText(this, "Please enter both phone and message",
Toast.LENGTH_SHORT).show();
        return;
    }

    try {
        SmsManager smsManager = SmsManager.getDefault();
        smsManager.sendTextMessage(phone, null, message, null, null);
        Toast.makeText(this, "SMS Sent Successfully", Toast.LENGTH_SHORT).show();
        binding.editMessage.setText("");
    } catch (Exception e) {
        Toast.makeText(this, "Failed to send SMS: " + e.getMessage(),
Toast.LENGTH_LONG).show();
        e.printStackTrace();
    }
}

@Override
protected void onResume() {
    super.onResume();
    MessageReceiver.addListener(this);
    IntentFilter filter = new IntentFilter("android.provider.Telephony.SMS_RECEIVED");
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        registerReceiver(messageReceiver, filter, RECEIVER_EXPORTED);
    } else {
        registerReceiver(messageReceiver, filter);
    }
}

@Override
protected void onPause() {
    super.onPause();
    unregisterReceiver(messageReceiver);
    MessageReceiver.addListener(null);
}

@Override
public void onMessageReceived(String message) {
    new AlertDialog.Builder(this)
        .setTitle("Incoming Message Alert")
        .setMessage(message)
        .setPositiveButton("Dismiss", null)

```

```
        .show();
    }
}
```

MessageReceiver.java

Path: app/src/main/java/com/example/alert_receiving_a_msg/MessageReceiver.java

```
package com.example.alert_receiving_a_msg;
```

```
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.telephony.SmsMessage;
import android.widget.Toast;
```

```
public class MessageReceiver extends BroadcastReceiver {
```

```
    public interface MessageListener {
        void onMessageReceived(String message);
    }
```

```
    private static MessageListener mListener;
```

```
    public static void bindListener(MessageListener listener) {
        mListener = listener;
    }
```

```
    @Override
```

```
    public void onReceive(Context context, Intent intent) {
        Bundle data = intent.getExtras();
        if (data != null) {
            Object[] pdus = (Object[]) data.get("pdus");
            String format = data.getString("format");
            if (pdus != null) {
                for (Object pdu : pdus) {
                    SmsMessage smsMessage = SmsMessage.createFromPdu((byte[]) pdu, format);
                    String messageBody = smsMessage.getMessageBody();

                    if (mListener != null) {
                        mListener.onMessageReceived(messageBody);
                    } else {

```

```

        Toast.makeText(context, "Message: " + messageBody,
        Toast.LENGTH_LONG).show();
    }
}
}
}
}
}
}
}

```

FirstFragment.java

Path: app/src/main/java/com/example/alert_receiving_a_msg/FirstFragment.java

```
package com.example.alert_receiving_a_msg;
```

```
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
```

```
import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;
```

```
import com.example.alert_receiving_a_msg.databinding.FragmentFirstBinding;
```

```
public class FirstFragment extends Fragment {
```

```
    private FragmentFirstBinding binding;
```

```
    @Override
```

```
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState
    ) {
```

```
        binding = FragmentFirstBinding.inflate(inflater, container, false);
        return binding.getRoot();
```

```
    }
```

```
    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
```

```

        binding.buttonFirst.setOnClickListener(v ->
            NavHostFragment.findNavController(FirstFragment.this)
                .navigate(R.id.action_FirstFragment_to_SecondFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }
}

```

SecondFragment.java

Path: app/src/main/java/com/example/alert_receiving_a_msg/SecondFragment.java

```
package com.example.alert_receiving_a_msg;
```

```
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
```

```
import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;
```

```
import com.example.alert_receiving_a_msg.databinding.FragmentSecondBinding;
```

```
public class SecondFragment extends Fragment {
```

```
    private FragmentSecondBinding binding;
```

```
    @Override
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState
    ) {
```

```
        binding = FragmentSecondBinding.inflate(inflater, container, false);
```

```

        return binding.getRoot();

    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        binding.buttonSecond.setOnClickListener(v ->
            NavHostFragment.findNavController(SecondFragment.this)
                .navigate(R.id.action_SecondFragment_to_FirstFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }
}

```

ExampleUnitTest.java

Path: app/src/test/java/com/example/alert_receiving_a_msg/ExampleUnitTest.java

```
package com.example.alert_receiving_a_msg;
```

```
import org.junit.Test;
```

```
import static org.junit.Assert.*;
```

```

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
public class ExampleUnitTest {
    @Test
    public void addition_isCorrect() {
        assertEquals(4, 2 + 2);
    }
}

```

ExampleInstrumentedTest.java

Path:

app/src/androidTest/java/com/example/alert_receiving_a_msg/ExampleInstrumentedTest.
java

```
package com.example.alert_receiving_a_msg;
```

```
import android.content.Context;
```

```
import androidx.test.platform.app.InstrumentationRegistry;  
import androidx.test.ext.junit.runners.AndroidJUnit4;
```

```
import org.junit.Test;  
import org.junit.runner.RunWith;
```

```
import static org.junit.Assert.*;
```

```
/**  
 * Instrumented test, which will execute on an Android device.  
 *  
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>  
 */  
@RunWith(AndroidJUnit4.class)  
public class ExampleInstrumentedTest {  
    @Test  
    public void useAppContext() {  
        // Context of the app under test.  
        Context appContext =  
InstrumentationRegistry.getInstrumentation().getTargetContext();  
        assertEquals("com.example.alert_receiving_a_msg", appContext.getPackageName());  
    }  
}
```

End of lab manual. Source listings generated from project files.